

PROJECT REPORT

Project Title: Task Management Console

Domain: Foundational Python Development & CLI Tools

Language/Tools: Python 3.x (Standard Library: `json`, `os`, `datetime`, `unittest`)

1. Problem Understanding

In daily personal and professional workflows, tracking tasks, priorities, and deadlines efficiently is critical. Manual tracking methods often lead to missed deadlines or disorganization. Furthermore, simple command-line applications often lack data validation, causing crashes when users input unexpected formats (such as invalid dates or empty text strings).

The primary requirement of this project was to design and implement a CLI-driven **Task Management Console** that:

- Encapsulates task attributes (ID, Title, Priority, Due Date, Status).
- Prevents data corruption through strict input validation (e.g., verifying `YYYY-MM-DD` date formats and allowed status keywords).
- Calculates dynamic properties such as overdue status without hardcoding.
- Ensures data persistence across sessions using JSON storage.

2. Proposed Approach & Architecture

To ensure the solution is clean, maintainable, and scalable, an **Object-Oriented Programming (OOP)** architecture was adopted. The code is structured into clear layers:

- **Data Representation Layer (`Task` Class):** Serves as the blueprint for an individual task item. It contains built-in instance methods and validation logic to guarantee that every task object in memory is valid.
- **Service / Business Logic Layer (`TaskManager` Class):** Manages the collection of tasks, handles unique auto-incrementing task IDs, runs filtering and keyword search routines, and calculates overall metrics for summary reporting.
- **Persistence Layer (`json` Integration):** Handles reading from and writing to `tasks.json` so user data remains intact after closing the program.
- **User Interface Layer (`main` & CLI Functions):** An interactive terminal loop (`while True`) that accepts user input, displays structured tables, catches exceptions gracefully, and displays status feedback.

3. Implementation & Technical Decisions

Key Technical Implementations:

- **Input Validation:** Built static/instance validation methods in the `Task` class. Dates are parsed using `datetime.strptime()` to confirm proper formatting (`YYYY-MM-DD`), while string fields are sanitized using `.strip()` and `.title()`.

- **Dynamic Overdue Tracking:** The method `is_overdue()` compares the task's due date string against the current system date (`datetime.now()`) using ISO standard string comparison. Completed tasks automatically bypass this check.
- **Robust File Handling:** The `load_from_file()` method checks for file existence using `os.path.exists()` before attempting to open `tasks.json`. Exception blocks capture `JSONDecodeError` or `IOError` cleanly to initialize a fresh ledger rather than crashing.

4. Testing & Verification

Testing was conducted using Python's built-in `unittest` framework in a separate script (`test_tasks.py`). Five primary test scenarios were implemented to verify system reliability:

Test Case	Scenario Type	Expected Result	Pass / Fail
TC1: Add Valid Task	Normal Input	Task added with ID #1, attributes match	PASS
TC2: Bad Date Format	Invalid Input	Throws <code>ValueError</code> for date "25-08-2026"	PASS
TC3: Empty Title	Boundary Case	Throws <code>ValueError</code> when title is whitespace " "	PASS
TC4: Overdue Logic	Logic Rule	Returns <code>True</code> for past pending task, <code>False</code> if completed	PASS
TC5: Delete Missing ID	Missing Data	Returns <code>False</code> when deleting non-existent ID #999	PASS

5. Challenges Encountered & Solutions

- **Challenge 1: First-Run File Crashing**
 - *Issue:* The application attempted to read `tasks.json` on the first launch before any tasks were created, raising a `FileNotFoundError`.
 - *Solution:* Added an explicit check using `os.path.exists(self.filename)` inside `load_from_file()` to handle initial startup cleanly.
- **Challenge 2: Inaccurate Overdue Flags for Finished Tasks**
 - *Issue:* Tasks marked as "Completed" were still being flagged as overdue if their original due date was in the past.

- *Solution:* Updated `is_overdue()` to explicitly verify `if self.status == "Completed"`:
`return False` prior to date comparisons.

6. Summary & Future Scope

The **Task Management Console** satisfies all foundational core Python development requirements, demonstrating OOP principles, file handling, error handling, lists/dictionaries, validation, and automated testing.

Future Improvements:

- Add category/tag grouping (e.g., Work, Personal, Study).
- Implement CSV export support for external reporting.
- Upgrade the terminal interface to a graphical user interface (GUI) using `tkinter`.